

# RPI Pico W Game Engine

## Documentation

## Components

### AIComponent

```
struct AIComponent
{
    std::function<void(World&, uint8_t, float)> think;
    std::function<void(World&, uint8_t, float)> decide;
    std::function<void(World&, uint8_t, float)> act;

    float memory[5];
};
```

### JumpComponent

```
struct JumpComponent
{
    float jumpPower = 0;
    bool isGrounded = false;
};
```

### MovementComponent

```
struct MovementComponent
{
    float speed = 0;
};
```

## TimerComponent

```
struct TimerComponent
{
    float currentTime = 0;
};
```

## TransformComponent

```
struct TransformComponent
{
    Vector2 currentPosition = {};
    Vector2 lastPosition = {};
    Vector2 currentSize = {};
};
```

## OnEventComponent

```
struct OnEventComponent
{
    std::unordered_map<EventType, std::function<void(const Event&)>> events;
};
```

## InputIntendComponent

```
struct InputIntendComponent
{
    int x = 0; //horizontal movement
    int y = 0; //vertical movement
    bool hasX = false;
    bool hasY = false;
    std::unordered_map<InputAction, bool> actions;
};
```

## InputMappingComponent

```
struct InputMappingComponent
{
    std::unordered_map<KEYCODE, std::pair<InputAction, int>> directionMapping;
    std::unordered_map<KEYCODE, InputAction> actionMapping;
};
```

## ColliderComponent

```
struct ColliderComponent
{
    Vector2 offset = {0,0};
    Vector2 size = {0,0};
    bool isTrigger;

    CollisionEntry currentCollisions[MAX_COLLISIONS];
};
```

## CollisionResponseComponent

```
struct CollisionResponseComponent
{
    std::function<void(World& world, uint8_t self, uint8_t other)> OnEnter;
    std::function<void(World& world, uint8_t self, uint8_t other)> OnStay;
    std::function<void(World& world, uint8_t self, uint8_t other)> OnExit;
};
```

## PhysicsComponent

```
struct PhysicsComponent
{
    bool useGravity;
    PhysicsType physicsType;
    Vector2 currentVelocity;
    bool facingLeft;
};
```

## PhysicsMaterialComponent

```
struct PhysicsMaterialComponent
{
    float friction = 0; //oncollision stay
    float bounciness = 0; //on collision enter
};
```

## AnimationComponent

```
struct AnimationComponent
{
    uint16_t currentAnimation = 0;
    uint16_t currentFrame = 0;

    float currentTime = 0.0f;
    int direction = 1;

    bool active = true;
};
```

## AnimationSetComponent

```
struct AnimationSetComponent
{
    uint8_t states[STATE_COUNT];
};
```

## CameraComponent

```
struct CameraComponent
{
    float currentZoom;
    float targetZoom;
    float zoomSpeed = 2;
};
```

## FollowTargetComponent

```
struct FollowTargetComponent
{
    TransformComponent* target;
};
```

## RectangleComponent

```
struct RectangleComponent
{
    uint16_t currentColor = 0;
    bool filled = false;
};
```

## RenderableComponent

```
struct RenderableComponent
{
    bool doRender;
    int layer;
    float parallaxFactor;
};
```

## SpriteComponent

```
struct SpriteComponent
{
    uint16_t spriteID = 0;
};
```

## TilemapComponent

```
struct TilemapComponent
{
    int width;
    int height;

    uint16_t tilesetId;    // reference to global Tilesset
    std::vector<uint8_t> tileIndices;
};
```

## UIButtonComponent

```
struct UIButtonComponent
{
    bool isSelected = false;

    uint16_t defaultColor;
    uint16_t selectedColor;
    std::function<void(GameManager& manager)> OnClicked
};
```

## UITextComponent

```
struct UITextComponent
{
    std::string text;
    uint16_t textColor;
};
```

# Systems

## AISystem

Runs during Update to process AI behaviour.

### Required Components:

- AIComponent

## TimerSystem

Runs during Update to increase the timer.

### Required Components:

- TimerComponent

## EventComponentSystem

Called by the EventManager to send the Events to all Entities.

### Required Components:

- OnEventComponent

## InputSystem

Runs during Update to log Hardware Inputs and produce input intends.

### Required Components:

- InputIntendComponent
- InputMappingComponent

## FollowMovementSystem

Runs during Update to update the position in the direction of the target.

### Required Components:

- TransformComponent
- MovementComponent
- FollowTargetComponent



## InputMovementSystem

Runs during Update to progress any User Inputs.

### Required Components:

- PhysicsComponent
- MovementComponent
- InputIntendComponent

## JumpSystem

Runs during Update to check if any Entity is supposed to Jump.

### Required Components:

- InputIntendComponent
- JumpComponent
- PhysicsComponent

## MovementSystem

Runs during Update before any positions are updated to log the position during the last frame.

### Required Components:

- TransformComponent
- MovementComponent

## CollisionSystem

Runs during Update after the Physic System Update to check for collisions.

### Required Components:

- TransformComponent
- ColliderComponent

## PhysicsSystem

Runs during Update. Once before the Collision System to update all positions and once after the Collision System to resolve all collisions.

### Required Components:

- TransformComponent
- PhysicsComponent

## AnimationSystem

Runs during Update to check all animation states.

### Required Components:

- AnimationComponent
- SpriteComponent

## CameraSystem

Runs during Update to adjust camera position and zoom.

### Required Components:

- CameraComponent

## ShapeRenderSystem

Runs during Render to render all Shape Entities.

### Required Components:

- RenderableComponent
- TransformComponent
- RectangleComponent

## SpriteRenderSystem

Runs during Render to render all Sprite Entities.

### Required Components:

- RenderableComponent
- TransformComponent
- SpriteComponent

## TilemapSystem

System runs once during the game initialization and creates the individual tile entities.

### Required Components:

- TilemapComponent

## UIButtonSystem

Run once in Init to register all Buttons to the System. Then it runs during Update to handle Button navigation.

### Required Components:

- RectangleComponent
- UIButtonComponent

## UIRenderSystem

Runs during Render to display all UI elements on top of the game.

### Required Components:

- TransformComponent
- UITextComponent or RectangleComponent

## UITimerTextSystem

Runs during Update to Update a UI Text with the current timer.

### Required Components:

- TimerComponent
- UITextComponent

# Managers

## AssetManager

Global Storage for Sprites, Animations and a Tileset.

## DebugManager

Static Debug System which logs parallel on the screen and in the terminal.

## ResourceManager

Static SD Card loading System to load files from the SD Card.

## EventManager

Static System to cast events. Listeners need to register itself to this manager.

## GameManager

Overarching Manager which holds the current game, is responsible to load the next game.

## World

Heart of each Game which holds all the entities and is responsible for their management.

# Factories

Factories are used to create Entities. There should be a game specific Factory for each game. For common Entities and for Prototyping there are Common Factories:

## CommonFactory

- Top Down Player
- Side Scroller Player
- Static Camera
- Follow Camera
- Static Wall

## CommonUIFactory

- Button
- Text
- TimerText

# How To

## Create Entity

An Entity is a minimal struct which holds a ComponentMask, a Tag and a isAlive flag. To set a new EntityID just call CreateEntity() in the current World. The World returns INVALID\_ENTITY if there is no available space.

```
struct Entity
{
    uint32_t mask;
    EntityTag tag = EntityTag::Default;

    bool isAlive = true;
};

From SDFactory.cpp:
uint8_t id = world.CreateEntity();
if (id == INVALID_ENTITY) return INVALID_ENTITY;
```

## Add Components

To add Individual Components to an Entity the mask needs to reflect the active Components. And then the Components need to be initiated.

From SDFactory.cpp:

```
world.entities[id].mask = TransformBit;

world.transforms[id] =
{
    .currentPosition = pos,
    .lastPosition = pos,
    .currentSize = size
};
```

## Add Sprite

To add a Sprite to an Entity first the Sprite File needs to be registered to the AssetManager. Then to Render the Sprite the Entity needs a Renderable and Sprite Component.

From SDFactory.cpp:

```
uint8_t spriteID = world.assets.AddSprite(path, size.x, size.y, index);

world.sprites[id] =
{
    .spriteID = spriteID
};
```

## Add Animation

Adding Animations work similar to adding Sprites. It requires the Entity to hold a Renderable, Sprites, Animation and AnimationSet Component.

From SDFactory.cpp:

```
uint8_t idle = world.assets.AddAnimation("idle.bin", 16, 16, 2, 4, AnimationMode::LOOP);
uint8_t walk = world.assets.AddAnimation("walking.bin", 16, 16, 4, 4, AnimationMode::LOOP);

world.animations[id] =
{
    .currentAnimation = idle,
    .currentFrame = 0,
    .currentTime = 0,
    .direction = 1,
    .active = true,
};

world.animSets[id].states[AnimState::Idle] = idle;
world.animSets[id].states[AnimState::Walk] = walk;
```

## Add AI Logic

To add AI Behaviour to an Entity fill the think, decide, act lambda function in the AI Component. Use the memory slots to transfer information between the functions.

From PongFactory.cpp:

```
    ai.memory[PongAI::BALL_ID] = ballID;

    ai.think = [this](World& world, uint8_t self, float deltaTime)
    {
        this->PaddleThink(world, self, deltaTime);
    };

void PongFactory::PaddleAct(World &world, uint8_t self, float deltaTime)
{
    auto& myIntend = world.inputIntends[self];
    auto& myAI = world.ai[self];

    myIntend.y = myAI.memory[PongAI::TARGET_DIR_Y];
}
```



## Add Tilemap

There are two steps needed to create a Tilemap. First create a Tilesheet by loading the spritesheet to the AssetManager and setting all solid tiles.

From SDFactory.cpp:

```
world.assets.CreateTilesheet("TilesetTest.bin", 32, 32, 5);  
world.assets.myTilesheet.solidTiles[3] = true;
```

Next is to set the actual tile map. In this case it is saved as a .txt file on the SD Card:

From SDFactory.cpp:

```
auto& tm = world.tilemaps[id];  
tm.width = 10;  
tm.height = 10;  
  
// Load Tilemap File  
File tilemap = ResourceManager::GetInstance().sdManager.Open("Tilemap.txt", FA_READ);  
std::string text = tilemap.Read();  
tilemap.Close();  
  
for (char c : text)  
{  
    if (c == '\n') continue;  
  
    int index = c - '0'; //get index with ASCII  
  
    tm.tileIndices.emplace_back(index); //fill with indices  
}
```

## Use Debugger

As the Debugger is a static Manager it can be simply called. It can hold 10 Logs at a time and will delete the oldest log message first. The Log function takes in a C style message as well as a Color (if none set it prints it in red). Press B to toggle the visibility of the Log.

From GameManager.cpp:

```
std::string msg = "DT: " + std::to_string(deltaTime);  
Debugger::GetInstance().Log(msg.c_str());
```